



CLICKUPFY

Guia completo do usuário

Planeje. Execute. Comprove.

Do ClickUp ao terminal e ao agente, com escopo, contexto e evidência.

Edição v1.1.1

Gerado em 30 de julho de 2026

Fonte editorial: docs/user/

Sumário

Guia completo do usuário	5
Leia online ou como ebook	5
Percurso pedagógico	5
O modelo mental em uma frase	6
Entenda o ClickUpfy	7
O problema que a ferramenta resolve	7
As quatro camadas	7
Perfis e projetos são coisas diferentes	8
Leitura compacta e leitura executável	8
Limites importantes	8
Instale o ClickUpfy	9
Requisitos	9
Instale pelo npm	9
Instale um executável standalone	9
Instale para desenvolvimento	10
Confirme a instalação	10
Configure perfis e credenciais	11
Crie a API key	11
Automatize o setup quando necessário	11
Entenda o arquivo local	11
Gerencie accounts	12
Troque o workspace associado	13
Remova um perfil	13
Prepare o primeiro projeto	14
O que você vai preparar	14
Valide o perfil	14
Descubra a hierarquia	14
Inicialize o agente	14
Confira no agente	15
Libere escrita	16
Primeira leitura de uma tarefa	16
Navegue pela hierarquia do ClickUp	17
A ordem dos recursos	17

Liste workspaces e Spaces	17
Liste Folders e Lists	17
Confira os status da List	18
Liste e busque tarefas	18
Escolha o menor escopo suficiente	18
Trabalhe com tarefas, subtarefas e checklists	20
Leia antes de alterar	20
Entenda a fila executável	20
Crie uma tarefa	20
Monte checklists verificáveis	21
Atualize campos	22
Registre progresso em comentários	22
Exclua apenas com alvo confirmado	22
Planeje Sprints e Sprint Points	23
Como o ClickUpfy reconhece uma Sprint	23
Liste os ciclos	23
Encontre a Sprint atual	23
Leia o relatório	23
Associe uma tarefa sem mover a List principal	24
Defina Sprint Points	24
Configure o MCP para Sprints	24
Registre tempo de trabalho	26
Consulte antes de iniciar	26
Inicie um registro	26
Encerre o registro	26
Use outro account em uma operação	26
Relação com comentários e status	27
Use as skills para agentes	28
O catálogo incluído	28
Inspeção antes de instalar	28
Instale no projeto	28
Crie uma issue sem implementá-la	29
Implemente uma issue existente	29
Use a skill diária	29
Prepare uma release	30

Conecte agentes pelo servidor MCP	31
Inicie manualmente	31
Ative read-only	31
Entenda o isolamento	31
Ferramentas de contexto e navegação	32
Use Markdown para contexto longo	33
Diagnostique a conexão	34
Consulte a referência do CLI	35
Sintaxe global	35
Configuração e identidade	35
Hierarquia	36
Tarefas	36
Checklists e comentários	39
Sprints	39
Time tracking	40
Agentes e MCP	40
Escolha o formato de saída	40
Ajuda é a referência da versão instalada	41
Resolva falhas comuns	42
O comando não foi encontrado	42
A API key foi recusada	42
O workspace está incorreto	42
Folder ou List não aparece	42
O status foi rejeitado	43
O MCP não inicia	43
O MCP recusou um ID	43
Uma ferramenta de escrita não aparece	43
A Sprint atual não foi encontrada	43
O item de checklist não mudou	44
O time entry já está em execução	44
A saída compacta não mostra um campo	44
Diagnóstico seguro	44

Guia completo do usuário



O ClickUpfy conecta o trabalho mantido no ClickUp ao terminal e aos agentes de código. Um único executável configura vários perfis, percorre a hierarquia do ClickUp, administra tarefas, checklists, Sprints, comentários e time tracking, instala skills no projeto e oferece as mesmas operações por MCP.

Este guia ensina essa jornada em sequência. Você começa instalando o CLI e associando uma API key a um workspace. Depois encontra os IDs do projeto, executa uma tarefa completa pelo terminal e prepara um MCP restrito para que o agente trabalhe somente na List autorizada.

Leia online ou como ebook

Estas páginas também formam o **ClickUpfy — Guia completo do usuário**. A [pasta do ebook](#) publica a versão vigente em PDF e EPUB e registra os hashes que comprovam a origem comum dos formatos.

O PDF favorece leitura, compartilhamento e impressão. O EPUB se adapta ao tamanho de fonte e ao leitor digital. Os dois são reconstruídos sempre que uma página desta documentação muda.

Percurso pedagógico

Siga esta ordem na primeira leitura:

1. [Entenda o ClickUpfy](#) e os limites entre ClickUp, CLI, MCP e skills.
2. [Instale o CLI](#) pelo npm ou por um executável standalone.
3. [Configure perfis e credenciais](#) sem colocar a API key no repositório.
4. [Prepare o primeiro projeto](#) e confira o escopo antes de permitir escrita.
5. [Navegue pela hierarquia](#) até encontrar Space, Folder, List e Sprint Folder.
6. [Trabalhe com tarefas e checklists](#), incluindo subtarefas, Markdown, datas e comentários.
7. [Planeje Sprints](#) e acompanhe avanço por tarefas e Sprint Points.
8. [Registre tempo](#) no item em execução.
9. [Instale e use as skills](#) que acompanham criação, implementação e release.
10. [Conecte um agente por MCP](#) com IDs fixos e opção read-only.
11. [Consulte a referência do CLI](#) para comandos, argumentos e formatos de saída.

12. [Resolva falhas comuns](#) de autenticação, escopo, status e integração.

O modelo mental em uma frase

O arquivo global guarda credenciais. O arquivo do projeto guarda somente o perfil e os IDs autorizados.

Essa divisão permite usar o mesmo ClickUpfy em vários projetos sem duplicar a API key. Um projeto pode apontar para a List de um produto, enquanto outro aponta para a List de um cliente. O MCP de cada raiz recusa IDs diferentes dos que foram fixados.

Comece por [entender o papel de cada camada](#).

Entenda o ClickUpfy

O problema que a ferramenta resolve

O ClickUp continua sendo a fonte do trabalho: tarefa, descrição, status, checklist, comentário, Sprint e registro de tempo permanecem no workspace. O ClickUpfy não cria um gerenciador paralelo. Ele oferece uma interface previsível para consultar e alterar essa fonte pelo terminal ou por um agente.

Sem essa camada, cada agente precisa descobrir como autenticar, quais endpoints usar, em qual List trabalhar e como representar uma tarefa grande. O ClickUpfy centraliza essas decisões em três contratos:

- o perfil local associa uma API key a um workspace
- o projeto fixa o perfil e a hierarquia autorizada no `.mcp.json`
- as skills descrevem quando ler, planejar, comentar, medir tempo e concluir.

As quatro camadas

ClickUp

É o sistema remoto e a fonte de verdade. O ClickUpfy usa a API pública para ler workspaces, Spaces, Folders, Lists, tarefas, checklists, comentários, Sprints e time entries. Permissões e ClickApps continuam sendo administrados no ClickUp.

CLI

É a interface humana e automatizável. A sintaxe segue `recurso ação`, como `task get`, `sprint current` e `comment create`. Sem `--json`, a saída é compacta. Com `--json`, o CLI preserva a resposta completa necessária para scripts e diagnóstico.

Servidor MCP

É a interface para agentes. Ele usa transporte stdio e expõe ferramentas com schemas explícitos. Uma configuração por projeto pode fixar account, workspace, Space, Folder, List e Sprint Folder. A List é obrigatória porque define o destino das consultas, buscas e criações de tarefa.

Skills

São instruções operacionais entregues junto ao pacote. Elas não substituem o MCP: orientam o agente a usar as ferramentas na ordem adequada, preservar contexto, registrar progresso e comprovar a conclusão.

Perfis e projetos são coisas diferentes

Um **account** do ClickUpfy é um perfil local. Ele contém um nome, a API key, o usuário autenticado e o workspace associado. O arquivo vive fora dos repositórios, em `~/ .promovaweb-clickupfy/ config.json`.

Um **projeto** é uma raiz de código que contém `.mcp.json`. Esse arquivo seleciona um account e fixa IDs da hierarquia. Ele pode ser versionado quando os IDs não forem considerados sensíveis pela equipe, pois nunca contém a API key.

O mesmo account pode servir mais de um projeto. Ambientes de clientes ou workspaces diferentes devem receber perfis separados.

Leitura compacta e leitura executável

`task get` não retorna apenas a tarefa original. O ClickUpfy monta uma propriedade `execution` que achata a tarefa principal, subtarefas de qualquer nível e checklist items em uma fila ordenada.

Cada item informa:

- uma `key` estável para a leitura atual
- o tipo do item
- a relação com o pai
- a profundidade na árvore
- o estado aberto ou concluído
- a ação equivalente no CLI e no MCP.

Essa fila ajuda o agente a selecionar um item pendente sem perder a hierarquia. Quando o contexto precisa ser lido como documento, `task get --markdown` produz metadados, descrição, resumo e checkboxes em um único texto.

Limites importantes

O ClickUpfy não cria uma Sprint nem habilita o Sprint ClickApp, porque a API pública não oferece um endpoint próprio para essas operações. Crie o Sprint Folder e as Sprints na interface do ClickUp e use o CLI para consulta, associação de tarefas e Sprint Points.

O CLI também não decide sozinho qual status representa conclusão. Use `list get <list-id>` para ler os status configurados na List. Skills de implementação devem escolher um status terminal existente, não inventar uma grafia.

Agora siga para a [instalação](#).

Instale o ClickUpfy

Requisitos

Para instalar pelo npm ou desenvolver, use Node.js 22.12.0 ou superior. Para usar um executável standalone, o Node.js não é necessário porque o runtime já está incorporado.

Você também precisa de uma API key pessoal do ClickUp com acesso a pelo menos um workspace. A chave só será usada na etapa de [configuração](#).

Instale pelo npm

O pacote oficial instala o comando global:

```
npm install --global @promovaweb/clickupfy
clickupfy --version
clickupfy --help
```

O launcher seleciona o executável compilado em Linux x64, macOS Intel e macOS Apple Silicon. Em outras plataformas, inclusive Windows, usa o build JavaScript incluído no pacote.

Uma versão válida na primeira linha confirma que o pacote e o launcher foram encontrados. Se o terminal não localizar `clickupfy`, confira o diretório global do npm:

```
npm prefix --global
npm bin --global
```

O segundo comando pode não existir em versões novas do npm. Nesse caso, o diretório de executáveis costuma ser `bin` dentro do prefixo em Linux e macOS.

Instale um executável standalone

Cada GitHub Release distribui archives para Linux x64, macOS Intel, macOS Apple Silicon e Windows x64. Baixe o archive da plataforma, extraia `clickupfy` ou `clickupfy.exe` e mova o arquivo para uma pasta presente no `PATH`.

No Linux ou macOS:

```
chmod +x clickupfy
./clickupfy --version
```

Antes de usar um binário baixado, confira o arquivo `SHA256SUMS` da mesma release. O nome e a versão do archive precisam corresponder à plataforma escolhida.

No macOS, o Gatekeeper pode impedir a abertura de um executável sem notificação. Confirme a origem no repositório oficial e siga a política de segurança da máquina. Não desative proteções globais para contornar o aviso.

Instale para desenvolvimento

Clone o repositório independente do ClickUpfy e execute:

```
npm install
npm run build
npm link
clickupfy --version
```

`npm link` aponta o comando global para o checkout atual. Use esse modo apenas quando precisar desenvolver ou validar uma mudança local. Para voltar ao pacote publicado, remova o link e reinstale a versão do npm.

Confirme a instalação

Execute três leituras sem credenciais:

```
clickupfy --version
clickupfy --help
clickupfy agent skill list
```

O catálogo deve listar `clickupfy-dev`, `clickup-issue-create`, `clickup-issue-implement` e `clickupfy-release`. A presença das skills confirma que os assets foram empacotados junto ao CLI.

Ainda não execute comandos remotos. Primeiro [configure um perfil](#).

Configure perfis e credenciais

Crie a API key

Use uma API key pessoal criada nas configurações do ClickUp. O ClickUpfy valida a chave antes de salvar, consulta o usuário autenticado e lista os workspaces permitidos.

Não cole a chave em issue, comentário, arquivo `.env` versionado, `.mcp.json` ou comando compartilhado no histórico da equipe. Em um terminal interativo, prefira o setup com prompt oculto:

```
clickupfy setup
```

Escolha um nome local para o perfil e selecione o workspace. O nome vira um identificador normalizado, como `promovaweb` ou `cliente-a`.

Automatize o setup quando necessário

Em uma automação isolada, o setup aceita parâmetros:

```
clickupfy setup \
  --api-key "pk_..." \
  --name "Promovaweb" \
  --workspace "123456" \
  --non-interactive
```

Evite colocar a chave diretamente em um script. Leia o valor de um secret manager ou de uma variável protegida e apague o ambiente temporário depois da execução.

Entenda o arquivo local

Por padrão, a configuração fica em:

```
~/.promovaweb-clickupfy/config.json
```

O diretório recebe permissão `0700` e o arquivo `0600`. A API key precisa estar no JSON porque o CLI autentica chamadas futuras. Essa proteção limita a leitura ao usuário local, mas não substitui os cuidados com backup, malware ou pastas sincronizadas.

O schema permite vários accounts:

```
{
  "version": 1,
  "activeAccount": "promovaweb",
  "accounts": {
    "promovaweb": {
      "name": "Promovaweb",
      "apiKey": "pk_...",
      "user": {
        "id": 123,
        "username": "Desenvolvedor"
      },
      "workspace": {
        "id": "456",
        "name": "Engenharia"
      },
      "createdAt": "2026-07-29T12:00:00.000Z",
      "updatedAt": "2026-07-29T12:00:00.000Z"
    }
  }
}
```

Gerencie accounts

Use comandos que mascaram a API key:

```
clickupfy account list
clickupfy account show
clickupfy status
clickupfy whoami
```

`status` mostra o caminho da configuração, o account resolvido e o workspace. `whoami` faz uma chamada autenticada e confirma se a chave ainda é válida.

Para trocar o perfil ativo:

```
clickupfy account use cliente-a
```

Para usar outro account em um único comando, sem alterar o ativo:

```
clickupfy --account cliente-a task get 86abc123
```

A variável `PROMOVAWEB_CLICKUPFY_ACCOUNT` oferece a mesma seleção em automações. A variável `PROMOVAWEB_CLICKUPFY_CONFIG` aponta para outro JSON e é útil em testes ou ambientes efêmeros.

Troque o workspace associado

Liste os workspaces autorizados e escolha um deles:

```
clickupfy workspace list
clickupfy workspace use 123456
```

Essa ação altera o workspace do account atual. Configurações de projeto que fixam `--workspace` precisam continuar correspondendo ao perfil.

Remova um perfil

```
clickupfy account remove cliente-antigo
```

O CLI pede confirmação. Em automação, `--yes` confirma sem prompt. A remoção apaga o perfil local, não revoga a API key no ClickUp. Revogue a chave no ClickUp quando ela não for mais necessária.

Com o perfil validado, prepare o [primeiro projeto](#).

Prepare o primeiro projeto

O que você vai preparar

Neste percurso, um repositório será ligado a uma List do ClickUp. O resultado é um `.mcp.json` que inicia o ClickUpfy com um account e IDs fixos, além das quatro skills instaladas no projeto.

Antes de escrever qualquer tarefa, você vai:

1. validar o perfil
2. descobrir Space, Folder e List
3. instalar o MCP em read-only para conferir o destino
4. liberar escrita somente depois da revisão.

Valide o perfil

```
clickupfy status  
clickupfy whoami
```

Confirme o nome do account, o usuário e o workspace. Se qualquer dado estiver errado, volte à [configuração](#).

Descubra a hierarquia

```
clickupfy workspace list  
clickupfy space list  
clickupfy folder list --space <space-id>  
clickupfy list list --folder <folder-id>
```

Uma List pode pertencer diretamente ao Space. Nesse caso:

```
clickupfy list list --space <space-id>
```

Guarde os IDs confirmados. Se o projeto usa Sprints, localize também o Sprint Folder que contém as Lists temporais.

Inicialize o agente

Na raiz do repositório:

```
clickupfy --account promovaweb agent init \  
  --workspace 123 \  
  --space 10 \  
  --folder 20 \  
  --list 30
```

`space` e `list` são obrigatórios. `Workspace` e `Folder` podem ser omitidos quando não forem necessários ao escopo. Acrescente `--sprint-folder <id>` se o projeto usa Sprints.

O comando instala as skills e mescla o servidor no `.mcp.json` existente sem remover outros servidores:

```
{  
  "mcpServers": {  
    "promovaweb-clickupfy": {  
      "command": "clickupfy",  
      "args": [  
        "mcp",  
        "serve",  
        "--account",  
        "promovaweb",  
        "--workspace",  
        "123",  
        "--space",  
        "10",  
        "--folder",  
        "20",  
        "--list",  
        "30"  
      ]  
    }  
  }  
}
```

Antes de abrir o cliente MCP pela primeira vez, acrescente `--read-only` ao fim de `args`. O `agent init` prepara o servidor completo. Essa edição consciente reduz a superfície durante a conferência inicial.

Confira no agente

Reinicie ou recarregue o cliente MCP para que ele leia a nova configuração. Peça uma consulta ao contexto:

```
Use clickupfy_mcp_context e mostre o perfil, o workspace e a hierarquia deste projeto.
```

Compare o resultado com os IDs anotados. Depois liste as tarefas:

```
Use clickupfy_tasks_list sem informar listId.
```

O servidor deve aplicar a List configurada. Uma tentativa de consultar outra List deve ser recusada.

Libere escrita

Quando o destino estiver correto, remova apenas `--read-only` da entrada gerenciada. Reabra o cliente MCP e confirme que as ferramentas de escrita aparecem em `tools/list`.

Crie uma tarefa de teste somente se o projeto e a equipe autorizarem a mutação. Caso contrário, use uma tarefa real já existente para validar leitura, comentários e checklists.

Primeira leitura de uma tarefa

No terminal:

```
clickupfy task get <task-id> --markdown
```

No agente:

```
Leia <task-id> com clickupfy_task_get usando markdown: true. Não altere a tarefa.
```

Confira descrição, subtarefas, checklists e fila executável. Agora você já pode aprofundar a [hierarquia](#) ou começar a [trabalhar com tarefas](#).

Navegue pela hierarquia do ClickUp

A ordem dos recursos

O ClickUp organiza o trabalho nesta cadeia:

```
Workspace
├── Space
│   ├── List sem Folder
│   └── Folder
│       └── List
```

Sprints também são Lists, mas vivem normalmente dentro de um Sprint Folder e possuem `start_date` e `due_date`.

O account já conhece um workspace. Por isso, `space list` não exige esse ID. A partir do Space, os comandos precisam do pai para evitar uma busca ambígua.

Liste workspaces e Spaces

```
clickupfy workspace list
clickupfy space list
clickupfy space list --archived
```

O asterisco na saída compacta indica o workspace associado ao account. Use `--archived` somente quando precisar localizar uma estrutura antiga.

Liste Folders e Lists

```
clickupfy folder list --space <space-id>
clickupfy list list --folder <folder-id>
```

Para Lists que pertencem diretamente ao Space:

```
clickupfy list list --space <space-id>
```

Folder e Space são alternativas no comando de Lists. Não informe os dois ao mesmo tempo.

Confira os status da List

```
clickupfy list get <list-id>
```

Essa leitura é importante antes de atualizar uma tarefa. Os nomes e tipos de status são configuráveis no ClickUp. Uma equipe pode usar `feito`, outra `concluído`, e uma terceira pode manter mais de um estado terminal.

Registre o ID da List e escolha um status terminal existente. Não assuma que `done` ou `complete` será aceito.

Liste e busque tarefas

```
clickupfy task list --list <list-id>
clickupfy task search --query "autenticação"
```

`task list` consulta uma List conhecida. `task search` procura no workspace, mas o MCP de projeto restringe a busca à List fixada para impedir vazamento de contexto entre projetos.

Use `--json` quando precisar inspecionar campos não exibidos na tabela:

```
clickupfy --json task list --list <list-id>
```

A opção global vem antes do grupo do comando.

Escolha o menor escopo suficiente

Um MCP de projeto sempre exige List. Space e Folder ajudam as ferramentas de navegação, mas não ampliam o destino de criação. Sprint Folder só deve ser configurado quando as Sprints do projeto estiverem dentro dele.

Para um projeto com List diretamente no Space:

```
clickupfy --account produto agent init \
  --space 10 \
  --list 30
```

Para um projeto com Folder e Sprints:

```
clickupfy --account produto agent init \  
  --space 10 \  
  --folder 20 \  
  --list 30 \  
  --sprint-folder 40
```

Continue em [tarefas e checklists](#).

Trabalhe com tarefas, subtarefas e checklists

Leia antes de alterar

Comece pela tarefa completa:

```
clickupfy task get <task-id> --markdown
```

Esse formato reúne metadados, descrição, subtarefas e checklist items em um documento. Para automação estruturada, use:

```
clickupfy task get <task-id> --json
```

Para obter somente a resposta original da API, sem a fila `execution`:

```
clickupfy task get <task-id> --raw
```

`--markdown`, `--json` e `--raw` são mutuamente exclusivos.

Entenda a fila executável

O objeto `execution` transforma a árvore em itens ordenados. Uma chave como `task:86abc123` identifica uma tarefa ou subtarefa. Uma chave como `checklist:check-1:item-1` identifica um item de checklist.

O resumo informa totais abertos e concluídos. Cada item inclui `parentKey` e `depth`, por isso um agente consegue trabalhar em uma subtarefa aninhada sem tratá-la como item independente.

A fila é uma projeção da leitura atual, não um segundo estado. Sempre releia a tarefa depois de uma mutação.

Crie uma tarefa

Use Markdown na descrição quando o ClickUp ClickApp correspondente estiver disponível:

```
clickupfy task create \  
  --list <list-id> \  
  --name "Implementar autenticação" \  
  --markdown-content "Adicionar o fluxo de login e os testes." \  
  --start-date 2026-08-01 \  
  --due-date 2026-08-05
```

Datas aceitam o formato `AAAA-MM-DD`. Antes de criar, confira o fuso e a política de datas da equipe.

Para criar uma subtarefa, informe o pai:

```
clickupfy task create \  
  --list <list-id> \  
  --name "Criar testes de autenticação" \  
  --parent <task-id> \  
  --markdown-content "Cobrir sucesso, credencial inválida e limite."
```

Subtarefas podem receber novas subtarefas. A leitura executável preserva todos os níveis.

Monte checklists verificáveis

Crie o checklist na tarefa e depois os itens:

```
clickupfy checklist create <task-id> --name "Validação"  
clickupfy checklist item-create <checklist-id> --name "Executar testes unitários"  
clickupfy checklist item-create <checklist-id> --name "Executar build"
```

Os itens são criados abertos. Marque um item somente depois de executar sua verificação:

```
clickupfy checklist set \  
  <task-id> <checklist-id> <item-id> \  
  --resolved
```

Para reabrir:

```
clickupfy checklist set \  
  <task-id> <checklist-id> <item-id> \  
  --open
```

O ClickUpfy confirma primeiro que o item pertence à tarefa, grava o novo estado e relê a tarefa. O comando só comunica sucesso quando a API devolve o valor esperado.

Atualize campos

```
clickupfy task update <task-id> --status "em andamento"
clickupfy task update <task-id> --priority 2
clickupfy task update <task-id> --start-date 2026-08-01
clickupfy task update <task-id> --due-date 2026-08-05
clickupfy task update <task-id> --points 5
```

As prioridades seguem a API do ClickUp:

VALOR	PRIORIDADE
1	urgente
2	alta
3	normal
4	baixa

Consulte `list get` antes de definir status. Para remover datas:

```
clickupfy task update <task-id> --clear-start-date
clickupfy task update <task-id> --clear-due-date
```

Registre progresso em comentários

```
clickupfy comment list --task <task-id>
clickupfy comment create \
  --task <task-id> \
  --text "Testes focais passaram. Iniciando a regressão."
```

Um comentário útil registra um evento verificável: início, impedimento, checkpoint de teste ou conclusão. Evite publicar raciocínio interno, secrets, logs extensos ou promessas sem evidência.

Exclua apenas com alvo confirmado

```
clickupfy task delete <task-id> --yes
```

A exclusão é permanente na interface do CLI e exige confirmação. Leia a tarefa, confira List e nome e confirme se a solicitação autoriza exclusão antes de usar `--yes`.

Quando a equipe trabalha por ciclos, continue em [Sprints e Sprint Points](#).

Planeje Sprints e Sprint Points

Como o ClickUpfy reconhece uma Sprint

No ClickUp, uma Sprint é uma List dentro de um Sprint Folder. O ClickUpfy considera Sprint uma List que possua `start_date` e `due_date`. Lists comuns do mesmo Folder ficam fora do resultado, salvo quando `--include-regular` for usado.

O CLI não cria Sprints nem habilita o Sprint ClickApp. Essas operações continuam na interface do ClickUp.

Liste os ciclos

```
clickupfy sprint list --folder <sprint-folder-id>
```

Para diagnóstico de um Folder misto:

```
clickupfy sprint list \  
  --folder <sprint-folder-id> \  
  --include-regular
```

Confira nome, ID, início e término. Um período ausente indica uma List comum ou uma Sprint ainda não configurada corretamente.

Encontre a Sprint atual

```
clickupfy sprint current --folder <sprint-folder-id>
```

O comando espera uma única Sprint ativa na data atual. Nenhuma Sprint ativa ou mais de uma sobreposta produz uma falha explícita. Corrija o calendário no ClickUp antes de automatizar a seleção.

Leia o relatório

```
clickupfy sprint get <sprint-id>  
clickupfy sprint tasks <sprint-id>  
clickupfy sprint tasks <sprint-id> --open-only
```

O relatório percorre todas as páginas e inclui tarefas concluídas. O avanço é calculado por quantidade de tarefas e por Sprint Points. Uma tarefa sem Points participa da quantidade de itens, mas não acrescenta peso ao cálculo por pontos.

Use `--open-only` para a fila de trabalho corrente, não para o relatório final da Sprint.

Associe uma tarefa sem mover a List principal

```
clickupfy sprint add-task <sprint-id> <task-id>
```

A API associa a tarefa à Sprint e preserva sua List principal. Isso permite manter backlog e ciclo como dimensões diferentes.

Para remover a associação:

```
clickupfy sprint remove-task <sprint-id> <task-id>
```

Essa ação não exclui a tarefa.

Defina Sprint Points

```
clickupfy sprint set-points <task-id> 5
```

O mesmo campo pode ser atualizado pelo grupo de tarefas:

```
clickupfy task update <task-id> --points 8
```

Use a escala adotada pela equipe. O ClickUpfy transmite o número, mas não define se ele representa complexidade, esforço ou tamanho relativo.

Configure o MCP para Sprints

Acrescente o Folder ao projeto:

```
clickupfy --account produto agent init \  
  --space 10 \  
  --folder 20 \  
  --list 30 \  
  --sprint-folder 40
```

As ferramentas de Sprint podem omitir `folderId` quando esse valor está fixado. Um ID diferente é recusado. Consultas permanecem disponíveis em read-only. Associação e Points aparecem apenas no servidor com escrita.

Depois do planejamento, aprenda a [registrar o tempo](#).

Registre tempo de trabalho

Consulte antes de iniciar

```
clickupfy time current
```

Essa leitura evita iniciar um segundo registro enquanto outro item permanece em execução. Se houver um time entry ativo, confirme a tarefa e decida se ele deve continuar ou ser encerrado.

Inicie um registro

```
clickupfy time start \  
  --task <task-id> \  
  --description "Implementação e testes"
```

Use uma descrição curta e observável. O registro pertence ao usuário autenticado pelo account selecionado.

Em um fluxo com agente, inicie o tempo somente quando a implementação realmente começar. Leitura inicial, esclarecimento e espera por autorização não devem ser registrados automaticamente sem uma regra explícita da equipe.

Encerre o registro

```
clickupfy time stop
```

Depois, consulte novamente:

```
clickupfy time current
```

A ausência de registro em execução confirma o encerramento.

Use outro account em uma operação

```
clickupfy --account cliente-a time current  
clickupfy --account cliente-a time start \  
  --task <task-id> \  
  --description "Correção e regressão"
```

Essa seleção não altera o account ativo. É útil quando dois projetos usam workspaces diferentes na mesma máquina.

Relação com comentários e status

Time tracking não muda status nem publica comentário. As três ações representam evidências diferentes:

- status indica a etapa no fluxo
- comentário registra um evento legível pela equipe
- time entry mede o período atribuído ao trabalho.

A skill de implementação coordena essas ações, mas cada mutação continua explícita e verificável.

Continue em [skills para agentes](#).

Use as skills para agentes

O catálogo incluído

O pacote distribui quatro skills:

SKILL	RESPONSABILIDADE
<code>clickupfy-dev</code>	Consulta e atualização do trabalho diário de software.
<code>clickup-issue-create</code>	Criação de tarefa, subtarefas e checklists sem executar a implementação.
<code>clickup-issue-implement</code>	Execução acompanhada por plano, comentários, tempo, testes e status.
<code>clickupfy-release</code>	Preparação e acompanhamento de versões e artefatos.

As skills são instruções. Elas usam o CLI ou o MCP disponível, mas não contêm credenciais.

Inspecciona antes de instalar

```
clickupfy agent skill list
clickupfy agent skill show clickupfy-dev
```

`list` apresenta o catálogo. `show` imprime a fonte empacotada da skill, útil para revisar permissões e fluxo antes da instalação.

Instale no projeto

Na raiz do repositório:

```
clickupfy agent skill install
```

As skills entram na pasta local compatível com o agente. Esse modo mantém a configuração próxima ao projeto e permite versionar as instruções.

Para instalar no catálogo global do usuário:

```
clickupfy agent skill install --global
```

Prefira a instalação local quando projetos usam versões ou regras diferentes.

Use `--force` para substituir uma skill já instalada somente depois de revisar as mudanças:

```
clickupfy agent skill install --force
```

Crie uma issue sem implementá-la

Peça explicitamente a skill de criação:

```
Use $clickup-issue-create para transformar esta solicitação em uma tarefa no ClickUp. Crie subtarefas e checklists de teste, mas não implemente o trabalho.
```

A skill deve obter os status da List, redigir a descrição em Markdown, criar a tarefa principal e materializar subtarefas e checklists recursivos. A criação não autoriza editar código nem marcar validações como concluídas.

Implemente uma issue existente

```
Use $clickup-issue-implement para executar a tarefa <task-id> até a validação final.
```

O fluxo lê a tarefa inteira, monta um plano visível, registra início, datas e time tracking quando aplicável, percorre subtarefas e checklist items e publica checkpoints. Um item de teste só é resolvido depois do comando correspondente passar.

A conclusão exige releitura do estado remoto, regressão aplicável e um status terminal existente na List.

Use a skill diária

`clickupfy-dev` é adequada para consultas e mutações pontuais:

```
Use $clickupfy-dev para ler a tarefa <task-id>, publicar o resultado dos testes e marcar apenas o checklist correspondente.
```

Escopo e autorização continuam limitados pelo pedido. A skill não transforma uma consulta em implementação completa.

Prepare uma release

`clickupfy-release` lê a documentação de release do próprio repositório e acompanha versão, changelog, validações e artefatos. Ela não deve ser instalada como regra genérica em projetos que apenas consomem o CLI.

Agora veja como essas skills acessam o ClickUp pelo [servidor MCP](#).

Conecte agentes pelo servidor MCP

Inicie manualmente

O servidor usa transporte stdio e reserva a saída padrão para as mensagens JSON-RPC trocadas com o cliente MCP:

```
clickupfy mcp serve --list 30
```

A List é obrigatória porque limita consultas e mutações ao destino do projeto. O comando abaixo também fixa os níveis superiores e o Sprint Folder:

```
clickupfy mcp serve \  
  --account promovaweb \  
  --workspace 123 \  
  --space 10 \  
  --folder 20 \  
  --list 30 \  
  --sprint-folder 40
```

Na prática, deixe o cliente MCP iniciar esse processo pelo `.mcp.json`.

Ative read-only

```
clickupfy mcp serve --list 30 --read-only
```

O modo read-only não registra ferramentas de escrita. Elas deixam de aparecer em `tools/list`, portanto a restrição é aplicada na superfície do servidor e não depende apenas da obediência do agente.

Use esse modo para descoberta, auditoria, demonstração ou qualquer conversa que não autorize alterações. Confira `tools/list` no cliente para confirmar que as ferramentas de escrita não foram registradas.

Entenda o isolamento

O MCP recebe IDs no processo. Quando uma ferramenta omite um ID, o servidor usa o valor fixado. Quando recebe um valor diferente, recusa a operação. Essa comparação protege estes níveis:

- account

- workspace
- Space
- Folder
- List
- Sprint Folder.

A busca de tarefas também permanece dentro da List do projeto. Assim, dois clientes MCP podem operar em paralelo sem compartilhar destino.

Ferramentas de contexto e navegação

`clickupfy_mcp_context` mostra perfil e hierarquia sem expor a API key. `clickupfy_list_get` retorna os status válidos da List.

Catálogo disponível em read-only

FERRAMENTA	FUNÇÃO
<code>clickupfy_mcp_context</code>	Mostra o account e os IDs fixados pelo projeto.
<code>clickupfy_accounts_list</code>	Lista accounts locais sem revelar API keys.
<code>clickupfy_whoami</code>	Valida o account e retorna o usuário autenticado.
<code>clickupfy_workspaces_list</code>	Lista workspaces autorizados.
<code>clickupfy_spaces_list</code>	Lista Spaces do workspace do account.
<code>clickupfy_folders_list</code>	Lista Folders do Space fixado ou informado.
<code>clickupfy_lists_list</code>	Lista Lists de um Folder ou diretamente de um Space.
<code>clickupfy_list_get</code>	Lê a List do projeto e seus status.
<code>clickupfy_tasks_list</code>	Lista tarefas da List autorizada.
<code>clickupfy_tasks_search</code>	Busca tarefas dentro da List autorizada.
<code>clickupfy_task_get</code>	Lê uma tarefa como resposta original, fila <code>execution</code> ou Markdown.
<code>clickupfy_comments_list</code>	Lê os comentários de uma tarefa.
<code>clickupfy_sprints_list</code>	Lista Sprints do Sprint Folder.
<code>clickupfy_sprint_current</code>	Obtém a Sprint ativa na data de referência.
<code>clickupfy_sprint_get</code>	Produz o relatório de uma Sprint.
<code>clickupfy_sprint_tasks</code>	Lista tarefas associadas à Sprint.

FERRAMENTA	FUNÇÃO
<code>clickupfy_time_current</code>	Consulta o time entry em execução.

Catálogo acrescentado no modo com escrita

FERRAMENTA	FUNÇÃO
<code>clickupfy_account_use</code>	Altera o account ativo quando o servidor não fixa um account.
<code>clickupfy_workspace_use</code>	Altera o workspace quando account e workspace não estão fixados.
<code>clickupfy_task_create</code>	Cria tarefa ou subtarefa na List autorizada.
<code>clickupfy_task_update</code>	Atualiza campos de uma tarefa.
<code>clickupfy_checklist_create</code>	Cria um checklist em uma tarefa.
<code>clickupfy_checklist_item_create</code>	Acrescenta um item ao checklist.
<code>clickupfy_checklist_item_set</code>	Marca ou reabre um item e confirma o novo estado.
<code>clickupfy_sprint_add_task</code>	Associa uma tarefa à Sprint.
<code>clickupfy_sprint_remove_task</code>	Remove a associação sem excluir a tarefa.
<code>clickupfy_sprint_set_points</code>	Define Sprint Points.
<code>clickupfy_task_delete</code>	Exclui uma tarefa mediante <code>confirm: true</code> .
<code>clickupfy_comment_create</code>	Publica um comentário na tarefa.
<code>clickupfy_time_start</code>	Inicia um time entry associado à tarefa.
<code>clickupfy_time_stop</code>	Encerra o time entry atual.

Na criação e na atualização, informe apenas os campos que devem ser enviados. O servidor recusa IDs que não correspondem ao escopo fixado. A skill de implementação consulta `clickupfy_time_current` antes de iniciar outro registro.

Use Markdown para contexto longo

`clickupfy_task_get` aceita `markdown: true`. Nesse modo, a ferramenta retorna um texto único, sem envolver o documento em uma string JSON escapada. Essa forma reduz ruído para agentes e preserva checkboxes hierárquicos.

Esta solicitação usa a leitura em Markdown e informa ao agente que nenhuma mutação está autorizada:

Leia a tarefa 86abc123 com `clickupfy_task_get` e `markdown: true`. Resuma o objetivo, liste os itens pendentes e não faça mutações.

Diagnostique a conexão

Se o cliente não listar ferramentas, siga a conexão desde o comando local até o recarregamento do processo:

1. execute o comando de `args` manualmente no terminal
2. confira se o account existe
3. confirme que `--list` possui valor
4. valide o JSON
5. reinicie o cliente MCP
6. leia os logs sem imprimir a API key.

A [referência do CLI](#) reúne os comandos e as flags usados para reproduzir o servidor manualmente quando o diagnóstico exigir uma comparação.

Consulte a referência do CLI

Sintaxe global

```
clickupfy [options] [command]
```

As opções globais selecionam o account e o formato da resposta antes que o CLI interprete o grupo e a ação:

OPÇÃO	EFEITO
<code>-V</code> , <code>--version</code>	Mostra a versão.
<code>--account <perfil></code>	Usa outro account somente nessa execução.
<code>--json</code>	Imprime a resposta completa em JSON.
<code>-h</code> , <code>--help</code>	Mostra ajuda.

Coloque as opções globais antes do grupo. No exemplo abaixo, a resposta de `task get` usa o account `cliente-a` e sai em JSON:

```
clickupfy --account cliente-a --json task get 86abc123
```

Configuração e identidade

```
clickupfy setup [options]
clickupfy status
clickupfy whoami
clickupfy account list
clickupfy account show [perfil]
clickupfy account use <perfil>
clickupfy account remove <perfil>
clickupfy workspace list
clickupfy workspace use <workspace-id>
```

O setup recebe a credencial, identifica o account local e associa o workspace. Estes parâmetros permitem executar o mesmo processo com ou sem prompts:

OPÇÃO	EFEITO
<code>--api-key <chave></code>	Informa a API key pessoal do ClickUp.

OPÇÃO	EFEITO
<code>--token <chave></code>	Mantém compatibilidade como alias de <code>--api-key</code> .
<code>--name <nome></code>	Define o nome local do account.
<code>--workspace <id></code>	Associa o account ao workspace indicado.
<code>--non-interactive</code>	Executa sem abrir prompts e exige os valores necessários por opção.

`account remove <perfil>` aceita `--yes` para confirmar a remoção sem prompt. As outras ações desse grupo não possuem opções próprias.

Hierarquia

```
clickupfy space list [--archived]
clickupfy folder list --space <id> [--archived]
clickupfy list list --folder <id> [--archived]
clickupfy list list --space <id> [--archived]
clickupfy list get <list-id>
```

`space list`, `folder list` e `list list` aceitam `--archived`. O comando `folder list` exige `--space <id>`. Em `list list`, informe `--folder <id>` ou `--space <id>`, pois os dois destinos são mutuamente exclusivos e a resposta precisa pertencer a um único nível da hierarquia.

Tarefas

```
clickupfy task list --list <list-id>
clickupfy task search --query <texto>
clickupfy task get <task-id>
clickupfy task get <task-id> --markdown
clickupfy task get <task-id> --raw
clickupfy task create [options]
clickupfy task update <task-id> [options]
clickupfy task delete <task-id> [--yes]
```

Filtros de leitura

COMANDO	OPÇÃO	EFEITO
<code>task list</code>	<code>--list <id></code>	Define a List consultada e é obrigatório.

COMANDO	OPÇÃO	EFEITO
<code>task list</code>	<code>--status <status...></code>	Restringe a resposta aos status informados.
<code>task list</code>	<code>--assignee <ids...></code>	Restringe a resposta aos responsáveis informados.
<code>task list</code>	<code>--include-closed</code>	Inclui tarefas concluídas.
<code>task list</code>	<code>--page <n></code>	Consulta uma página, começando em zero.
<code>task search</code>	<code>--query <texto></code>	Procura no nome, na descrição ou no ID.
<code>task search</code>	<code>--status <status...></code>	Restringe a busca aos status informados.
<code>task search</code>	<code>--assignee <ids...></code>	Restringe a busca aos responsáveis informados.
<code>task search</code>	<code>--include-closed</code>	Inclui tarefas concluídas.
<code>task search</code>	<code>--max-pages <n></code>	Limita a paginação consultada entre uma e 100 páginas.
<code>task get</code>	<code>--raw</code>	Retorna a resposta original do ClickUp.
<code>task get</code>	<code>--markdown</code>	Concatena tarefa, subtarefas e checklists em Markdown.

`--raw` e `--markdown` não podem ser usados juntos. A opção global `--json` também é incompatível com esses dois formatos, e o CLI encerra a execução com uma mensagem de uso quando encontra a combinação.

Campos de criação

OPÇÃO	EFEITO
<code>--list <id></code>	Define a List de destino e é obrigatório.
<code>--name <nome></code>	Define o nome e é obrigatório.
<code>--description <texto></code>	Envia uma descrição em texto.
<code>--markdown-content <markdown></code>	Envia uma descrição preservada como Markdown.
<code>--status <status></code>	Define o status inicial.
<code>--priority <n></code>	Define prioridade de 1 a 4.

OPÇÃO	EFEITO
<code>--assignee <ids...></code>	Informa IDs numéricos dos responsáveis.
<code>--parent <task-id></code>	Cria uma subtarefa ligada ao item informado.
<code>--start-date <AAAA-MM-DD></code>	Define a data de início.
<code>--due-date <AAAA-MM-DD></code>	Define a data de entrega.
<code>--points <n></code>	Define Sprint Points com número igual ou maior que zero.

Escolha `--description` para texto ou `--markdown-content` para Markdown. Evite enviar os dois campos na mesma criação para não deixar a representação da descrição ambígua.

Campos de atualização

OPÇÃO	EFEITO
<code>--name <nome></code>	Altera o nome.
<code>--description <texto></code>	Substitui a descrição por texto.
<code>--markdown-content <markdown></code>	Substitui a descrição por Markdown.
<code>--status <status></code>	Altera o status.
<code>--priority <n></code>	Altera a prioridade de 1 a 4.
<code>--start-date <AAAA-MM-DD></code>	Altera a data de início.
<code>--clear-start-date</code>	Remove a data de início.
<code>--due-date <AAAA-MM-DD></code>	Altera a data de entrega.
<code>--clear-due-date</code>	Remove a data de entrega.
<code>--points <n></code>	Altera os Sprint Points.

O comando exige pelo menos um campo. Quando uma data e sua opção `--clear-*` aparecem na mesma execução, a remoção prevalece. `task delete` usa `--yes` para confirmar a exclusão sem prompt.

Depois de escolher os campos, consulte a ajuda da versão instalada para comparar a referência com o contrato executável:

```
clickupfy task create --help
clickupfy task update --help
```

Checklists e comentários

```
clickupfy checklist create <task-id> --name <nome>
clickupfy checklist item-create <checklist-id> --name <nome>
clickupfy checklist set \
  <task-id> <checklist-id> <item-id> \
  --resolved
clickupfy checklist set \
  <task-id> <checklist-id> <item-id> \
  --open
clickupfy comment list --task <task-id>
clickupfy comment create --task <task-id> --text <texto>
```

`checklist item-create` aceita `--assignee <id>` para associar o item a um responsável. Em `checklist set`, escolha exatamente `--resolved` ou `--open` para que a releitura confirme o estado pretendido. `comment create` aceita `--notify-all` quando a atualização precisa notificar os participantes da tarefa.

Sprints

```
clickupfy sprint list --folder <folder-id>
clickupfy sprint current --folder <folder-id>
clickupfy sprint get <sprint-id>
clickupfy sprint tasks <sprint-id> [--open-only]
clickupfy sprint add-task <sprint-id> <task-id>
clickupfy sprint remove-task <sprint-id> <task-id>
clickupfy sprint set-points <task-id> <points>
```

Os comandos de Sprint aceitam opções de calendário, inclusão e estado para separar uma List comum do ciclo que será analisado:

COMANDO	OPÇÃO	EFEITO
<code>sprint list</code>	<code>--archived</code>	Inclui Lists arquivadas.
<code>sprint list</code>	<code>--include-regular</code>	Inclui Lists sem período do Sprint Folder.
<code>sprint list</code>	<code>--at <AAAA-MM-DD></code>	Calcula o estado do ciclo na data informada.
<code>sprint current</code>	<code>--at <AAAA-MM-DD></code>	Procura a Sprint ativa na data informada.

COMANDO	OPÇÃO	EFEITO
<code>sprint get</code>	<code>--at <AAAA-MM-DD></code>	Calcula o relatório na data informada.
<code>sprint tasks</code>	<code>--open-only</code>	Omite tarefas concluídas.

`sprint list` e `sprint current` exigem `--folder <id>` para limitar a busca ao Sprint Folder selecionado.

Time tracking

```
clickupfy time current
clickupfy time start --task <task-id> [--description <texto>]
clickupfy time stop
```

Agentes e MCP

```
clickupfy agent skill list
clickupfy agent skill show <nome>
clickupfy agent skill install [--global] [--force]
clickupfy agent init [options]
clickupfy mcp serve --list <id> [options]
```

`agent skill install` aceita uma lista opcional de skills. As opções `--global`, `--target <pasta>` e `--force` controlam destino e substituição, permitindo conferir a pasta instalada sem alterar outros catálogos.

`agent init` exige `--space <id>` e `--list <id>`. Ele também aceita `--global`, `--workspace <id>`, `--folder <id>`, `--sprint-folder <id>` e `--force`.

`mcp serve` exige `--list <id>` e aceita `--account <perfil>`, `--workspace <id>`, `--space <id>`, `--folder <id>`, `--sprint-folder <id>` e `--read-only`.

Escolha o formato de saída

A tabela compacta é adequada para leitura humana. Use `--json` quando um script precisar de campos completos ou quando a tabela não mostrar um detalhe da API.

Use `task get --markdown` para contexto textual. Use `--raw` apenas quando precisar comparar a resposta original do ClickUp com a projeção do ClickUpfy.

Ajuda é a referência da versão instalada

Esta documentação explica o contrato estável. Para flags exatas da versão presente na máquina:

```
clickupfy --help
clickupfy <grupo> --help
clickupfy <grupo> <ação> --help
```

Quando a ajuda estiver correta, mas a execução falhar, continue no guia de [solução de problemas](#) para conferir instalação, credencial, escopo e resposta da API.

Resolva falhas comuns

O comando não foi encontrado

Execute:

```
npm prefix --global
```

Confirme se a pasta de executáveis globais está no `PATH`. Em instalação standalone, confira permissão de execução e o nome do arquivo.

A API key foi recusada

Use:

```
clickupfy whoami
```

Se falhar, gere ou confira a chave no ClickUp e execute `clickupfy setup` novamente. Uma chave revogada não pode ser recuperada pelo ClickUpfy.

Nunca publique a chave para pedir suporte. Informe apenas o account, o workspace mascarado quando necessário e a mensagem de erro sem headers.

O workspace está incorreto

```
clickupfy workspace list  
clickupfy workspace use <workspace-id>  
clickupfy status
```

Se o MCP fixa `--workspace`, atualize a configuração do projeto para o mesmo ID ou selecione o account correto.

Folder ou List não aparece

Confirme o Space e tente incluir arquivados:

```
clickupfy folder list --space <space-id> --archived  
clickupfy list list --folder <folder-id> --archived
```

Para List sem Folder, use `--space`. Permissões da API key também podem ocultar recursos.

O status foi rejeitado

```
clickupfy list get <list-id>
```

Copie a grafia de um status retornado. Status são configuráveis por List.

O MCP não inicia

Execute manualmente o comando registrado no `.mcp.json`. Os motivos mais comuns são:

- `clickupfy` ausente no `PATH` do cliente
- JSON inválido
- account inexistente
- `--list` sem valor
- workspace fixado diferente do perfil
- cliente não reiniciado depois da edição.

O MCP recusou um ID

Essa recusa é esperada quando a ferramenta recebe um ID diferente do escopo do projeto. Consulte:

```
clickupfy_mcp_context
```

Use o destino fixado ou altere o `.mcp.json` conscientemente. Não contorne a proteção repetindo a chamada com outro recurso.

Uma ferramenta de escrita não aparece

O servidor pode estar em read-only. Confira os argumentos. Remover `--read-only` amplia as permissões do cliente e deve ser uma decisão explícita.

A Sprint atual não foi encontrada

Liste o Folder com períodos:

```
clickupfy sprint list --folder <sprint-folder-id> --include-regular
```

Confirme se existe exatamente uma Sprint cujo período inclui a data atual. Corrija datas ou sobreposição na interface do ClickUp.

O item de checklist não mudou

Releia a tarefa:

```
clickupfy task get <task-id> --json
```

Confirme task ID, checklist ID e item ID. O comando `checklist set` rejeita um item que não pertença à tarefa informada.

O time entry já está em execução

```
clickupfy time current  
clickupfy time stop
```

Confira o item antes de encerrar. Depois inicie o registro correto.

A saída compacta não mostra um campo

Repita com `--json`:

```
clickupfy --json task get <task-id>
```

Para uma tarefa longa consumida por agente, prefira `--markdown`.

Diagnóstico seguro

Ao relatar um problema, inclua:

- versão de `clickupfy --version`
- plataforma e forma de instalação
- grupo e ação executados
- mensagem de erro
- se o modo era CLI ou MCP
- escopo sem credenciais.

Remova API keys, tokens, headers, conteúdo de clientes e dados pessoais.

Volte ao [início do guia](#) ou consulte a [referência do CLI](#).